Client1

VS Code > OOPs Assignment > bank-management-system > src > client1.cpp

```cpp
cpp
#include "bank.h"
#include <iostream>
using namespace std;

int main() {
    cout << "\n========== CLIENT 1 OPERATIONS ==========\n\n";

    // Simulating client data (in real system, this would be loaded from file)
    Bank client1("Ali Ahmed", "ali123", "3520112345671", "ACC001", 5000.00);

    string username, password;

    cout << "Client 1 Login\n";
    cout << "Username: ";
    cin >> username;

    cout << "Password: ";
    cin >> password;

    cout << "\n--- Step 1: View Account ---\n";
    client1.viewAccount(username, password);

    cout << "\n--- Step 2: Deposit Amount ---\n";
    double depositAmount;
    cout << "Enter amount to deposit: Rs. ";
    cin >> depositAmount;
    client1.deposit(depositAmount, username, password);

    cout << "\n--- Step 3: View Account After Deposit ---\n";
    client1.viewAccount(username, password);

    cout << "\n--- Step 4: Attempt to Withdraw More Than Balance ---\n";
    double withdrawAmount;
    cout << "Enter amount to withdraw (more than balance): Rs. ";
    cin >> withdrawAmount;
    client1.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 5: Withdraw Valid Amount ---\n";
    cout << "Enter amount to withdraw (valid amount): Rs. ";
    cin >> withdrawAmount;
    client1.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 6: Final Account View ---\n";
    client1.viewAccount(username, password);

    cout << "\nClient 1 operations completed.\n";

    return 0;
```

Client2

VS Code > OOPs Assignment > bank-management-system > src > client2.cpp

```cpp
cpp
#include "bank.h"
#include <iostream>
using namespace std;

int main() {
    cout << "\n========== CLIENT 2 OPERATIONS ==========\n\n";

    // Simulating client data
    Bank client2("Sara Khan", "sara456", "3520112345672", "ACC002", 7500.00);

    string username, password;

    cout << "Client 2 Login\n";
    cout << "Username: ";
    cin >> username;

    cout << "Password: ";
    cin >> password;

    cout << "\n--- Step 1: View Account ---\n";
    client2.viewAccount(username, password);

    cout << "\n--- Step 2: Deposit Amount ---\n";
    double depositAmount;
    cout << "Enter amount to deposit: Rs. ";
    cin >> depositAmount;
    client2.deposit(depositAmount, username, password);

    cout << "\n--- Step 3: View Account After Deposit ---\n";
    client2.viewAccount(username, password);

    cout << "\n--- Step 4: Attempt to Withdraw More Than Balance ---\n";
    double withdrawAmount;
    cout << "Enter amount to withdraw (more than balance): Rs. ";
    cin >> withdrawAmount;
    client2.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 5: Withdraw Valid Amount ---\n";
    cout << "Enter amount to withdraw (valid amount): Rs. ";
    cin >> withdrawAmount;
    client2.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 6: Final Account View ---\n";
    client2.viewAccount(username, password);

    cout << "\nClient 2 operations completed.\n";

    return 0;
```

Client3

VS Code > OOPs Assignment > bank-management-system > src > client3.cpp >

```cpp
cpp
#include "bank.h"
#include <iostream>
using namespace std;

int main() {
    cout << "\n========== CLIENT 3 OPERATIONS ==========\n\n";

    // Simulating client data
    Bank client3("Bilal Ahmed", "bilal789", "3520112345673", "ACC003", 3000.00);

    string username, password;

    cout << "Client 3 Login\n";
    cout << "Username: ";
    cin >> username;

    cout << "Password: ";
    cin >> password;

    cout << "\n--- Step 1: View Account ---\n";
    client3.viewAccount(username, password);

    cout << "\n--- Step 2: Deposit Amount ---\n";
    double depositAmount;
    cout << "Enter amount to deposit: Rs. ";
    cin >> depositAmount;
    client3.deposit(depositAmount, username, password);

    cout << "\n--- Step 3: View Account After Deposit ---\n";
    client3.viewAccount(username, password);

    cout << "\n--- Step 4: Attempt to Withdraw More Than Balance ---\n";
    double withdrawAmount;
    cout << "Enter amount to withdraw (more than balance): Rs. ";
    cin >> withdrawAmount;
    client3.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 5: Withdraw Valid Amount ---\n";
    cout << "Enter amount to withdraw (valid amount): Rs. ";
    cin >> withdrawAmount;
    client3.withdraw(withdrawAmount, username, password);

    cout << "\n--- Step 6: Final Account View ---\n";
    client3.viewAccount(username, password);

    cout << "\nClient 3 operations completed.\n";

    return 0;
```

Bank.h

VS Code > OOPs Assignment > bank-management-system > src > bank.h > ...

```cpp
cpp
#ifndef BANK_H
#define BANK_H

#include <iostream>
#include <string>
using namespace std;

class Bank {
private:
    string clientName;
    string password;
    string clientCNIC;
    string accountNumber;
    double cashInAccount;

    // Static manager credentials (single for all clients)
    static string managerUsername;
    static string managerPassword;

public:
    // Constructors
    Bank();
    Bank(string name, string pass, string cnic, string accNo, double cash);

    // Static methods for manager verification
    static bool verifyManager(string username, string password);
    static void setManagerCredentials(string username, string password);

    // Client operations
    void viewAccount(string username, string pass);
    bool changeName(string cnic, string newName);
    void deposit(double amount, string username, string pass);
    bool withdraw(double amount, string username, string pass);

    // Getter methods for manager
    string getClientName() const { return clientName; }
    string getClientCNIC() const { return clientCNIC; }
    string getAccountNumber() const { return accountNumber; }
    double getCashInAccount() const { return cashInAccount; }

    // Utility functions
    static string getHiddenPassword();
    static bool isCNICUnique(string cnic, Bank* clients, int count);
    static bool isAccountUnique(string accNo, Bank* clients, int count);
};

#endif // BANK_H
```

Bank.cpp

VS Code > OOPs Assignment > bank-management-system > src > bank.cpp

```cpp
#include "bank.h"
#include <conio.h>  // For _getch()
#include <iomanip>
#include <cctype>

// Initialize static members
string Bank::managerUsername = "admin";
string Bank::managerPassword = "admin123";

// Constructors
Bank::Bank() {
    clientName = "";
    password = "";
    clientCNIC = "";
    accountNumber = "";
    cashInAccount = 0.0;
}

Bank::Bank(string name, string pass, string cnic, string accNo, double cash) {
    clientName = name;
    password = pass;
    clientCNIC = cnic;
    accountNumber = accNo;
    cashInAccount = cash;
}

// Static methods
bool Bank::verifyManager(string username, string password) {
    return (username == managerUsername && password == managerPassword);
}

void Bank::setManagerCredentials(string username, string password) {
    managerUsername = username;
    managerPassword = password;
}

// Get hidden password input (shows * instead of characters)
string Bank::getHiddenPassword() {
    string pass = "";
    char ch;

    cout << "Enter Password: ";

    while (true) {
        ch = _getch();

        if (ch == 13) { // Enter key
            cout << endl;
            break;
        } else if (ch == 8) { // Backspace
            if (pass.length() > 0) {
                cout << "\b \b";
                pass.pop_back();
            }
        } else {
            pass += ch;
            cout << "*";
        }
    }

    return pass;
}

// Check if CNIC is unique
bool Bank::isCNICUnique(string cnic, Bank* clients, int count) {
    for (int i = 0; i < count; i++) {
        if (clients[i].clientCNIC == cnic) {
            return false;
        }
    }
```

VS Code > OOPs Assignment > bank-management-system > src > bank.cpp >

```cpp
// Check if Account Number is unique
bool Bank::isAccountUnique(string accNo, Bank* clients, int count) {
    for (int i = 0; i < count; i++) {
        if (clients[i].accountNumber == accNo) {
            return false;
        }
    }
    return true;
}

// View account details (requires verification)
void Bank::viewAccount(string username, string pass) {
    if (username == clientName && pass == password) {
        cout << "\n========== ACCOUNT DETAILS ==========" << endl;
        cout << "Client Name: " << clientName << endl;
        cout << "CNIC: " << clientCNIC << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Current Balance: Rs. " << fixed << setprecision(2) << cashInAccount << endl;
        cout << "=====================================" << endl;
    } else {
        cout << "Error: Invalid username or password!" << endl;
    }
}

// Change name after CNIC verification
bool Bank::changeName(string cnic, string newName) {
    if (cnic == clientCNIC) {
        clientName = newName;
        cout << "Name changed successfully to: " << clientName << endl;
        return true;
    } else {
        cout << "Error: CNIC verification failed!" << endl;
        return false;
    }
}

// Deposit amount (requires verification)
void Bank::deposit(double amount, string username, string pass) {
    if (username == clientName && pass == password) {
        if (amount > 0) {
            cashInAccount += amount;
            cout << "Deposit successful! New balance: Rs. " << fixed << setprecision(2) << cashInAccount << endl;
        } else {
            cout << "Error: Invalid deposit amount!" << endl;
        }
    } else {
        cout << "Error: Invalid username or password!" << endl;
    }
}

// Withdraw amount (requires verification)
bool Bank::withdraw(double amount, string username, string pass) {
    if (username == clientName && pass == password) {
        if (amount > cashInAccount) {
            cout << "Invalid Operation: Insufficient balance!" << endl;
            cout << "Your current balance: Rs. " << fixed << setprecision(2) << cashInAccount << endl;
            return false;
        } else if (amount <= 0) {
            cout << "Error: Invalid withdrawal amount!" << endl;
            return false;
        } else {
            cashInAccount -= amount;
            cout << "Withdrawal successful! Amount withdrawn: Rs. " << fixed << setprecision(2) << amount << endl;
            cout << "Remaining balance: Rs. " << fixed << setprecision(2) << cashInAccount << endl;
            return true;
        }
    } else {
        cout << "Error: Invalid username or password!" << endl;
        return false;
    }
```

Manager.cpp

VS Code > OOPs Assignment > bank-management-system > src > manager.cpp >

```cpp
cpp
#include "bank.h"
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << "\n========== BANK MANAGEMENT SYSTEM ==========\n";
    cout << "============== MANAGER LOGIN ==============\n\n";

    // Manager authentication
    string username, password;
    cout << "Enter Manager Username: ";
    cin >> username;

    password = Bank::getHiddenPassword();

    if (!Bank::verifyManager(username, password)) {
        cout << "\nACCESS DENIED: Invalid manager credentials!\n";
        cout << "Program terminated.\n";
        return 1;
    }

    cout << "\nACCESS GRANTED: Welcome Manager!\n";
    cout << "==========================================\n\n";

    // Create array to store 3 clients
    Bank clients[3];
    int clientCount = 0;

    // Create 3 clients
    cout << "========== CREATING CLIENT ACCOUNTS ==========\n\n";

    for (int i = 0; i < 3; i++) {
        string name, pass, cnic, accNo;
        double initialCash;

        cout << "--- Client " << (i + 1) << " Registration ---\n";
        cout << "Enter Client Name: ";
        cin.ignore();
        getline(cin, name);

        cout << "Set Password: ";
        cin >> pass;

        // Get unique CNIC
        do {
            cout << "Enter CNIC (without dashes): ";
            cin >> cnic;
```

VS Code > OOPs Assignment > bank-management-system > src > manager.cpp > ...

```cpp
int main() {
    for (int i = 0; i < 3; i++) {
        do {

            if (!Bank::isCNICUnique(cnic, clients, clientCount)) {
                cout << "Error: CNIC must be unique! Try again.\n";
            }
        } while (!Bank::isCNICUnique(cnic, clients, clientCount));

        // Get unique Account Number
        do {
            cout << "Enter Account Number: ";
            cin >> accNo;

            if (!Bank::isAccountUnique(accNo, clients, clientCount)) {
                cout << "Error: Account Number must be unique! Try again.\n";
            }
        } while (!Bank::isAccountUnique(accNo, clients, clientCount));

        cout << "Enter Initial Deposit: Rs. ";
        cin >> initialCash;

        // Create client object
        clients[clientCount] = Bank(name, pass, cnic, accNo, initialCash);
        clientCount++;

        cout << "Client " << (i + 1) << " created successfully!\n\n";
    }

    cout << "\n========== ALL CLIENTS CREATED ==========\n";
    cout << "Press any key to continue...\n";
    cin.get();

    cout << "\n\n========== VIEWING ALL CLIENT DATA ==========\n\n";

    // Manager views all client data
    for (int i = 0; i < 3; i++) {
        cout << "--- Client " << (i + 1) << " Information ---\n";
        cout << "Name: " << clients[i].getClientName() << endl;
        cout << "CNIC: " << clients[i].getClientCNIC() << endl;
        cout << "Account Number: " << clients[i].getAccountNumber() << endl;
        cout << "Balance: Rs. " << fixed << setprecision(2) << clients[i].getCashInAccount() << endl;
        cout << "------------------------\n\n";
    }

    cout << "\nManager operations completed. Now clients will perform their operations.\n";
    cout << "Please run client1.cpp, client2.cpp, and client3.cpp separately.\n\n";

    return 0;
```

BANK MANAGEMENT SYSTEM - OOP ASSIGNMENT SOLUTION

---

4. SAMPLE OUTPUT

MANAGER OUTPUT (manager.cpp):

===== BANK MANAGEMENT SYSTEM =====
========= MANAGER LOGIN ==========

Enter Manager Username: admin
Enter Password: *******

ACCESS GRANTED: Welcome Manager!
==========
===== CREATING CLIENT ACCOUNTS ======

--- Client 1 Registration ---
Enter Client Name: Ali Ahmed
Set Password: ali123
Enter CNIC (without dashes): 3520112345671
Enter Account Number: ACC001
Enter Initial Deposit: Rs. 5000
Client 1 created successfully!

--- Client 2 Registration ---
Enter Client Name: Sara Khan
Set Password: sara456
Enter CNIC (without dashes): 3520112345672
Enter Account Number: ACC002
Enter Initial Deposit: Rs. 7500
Client 2 created successfully!

--- Client 3 Registration ---
Enter Client Name: Bilal Ahmed
Set Password: bilal789
Enter CNIC (without dashes): 3520112345673
Enter Account Number: ACC003
Enter Initial Deposit: Rs. 3000
Client 3 created successfully!

====== VIEWING ALL CLIENT DATA ======

--- Client 1 Information ---
Name: Ali Ahmed
CNIC: 3520112345671
Account Number: ACC001
Balance: Rs. 5000.00

--- Client 2 Information ---
Name: Sara Khan
CNIC: 3520112345672
Account Number: ACC002
Balance: Rs. 7500.00

--- Client 3 Information ---
Name: Bilal Ahmed
CNIC: 3520112345673
Account Number: ACC003
Balance: Rs. 3000.00

CLIENT 1 OUTPUT (client1.cpp):

====== CLIENT 1 OPERATIONS =========

Client 1 Login
Username: Khadija
Password: khadija@123

--- Step 1: View Account ---

========= ACCOUNT DETAILS =========
Client Name: Khadija

CNIC: 33401-0741908-0
Account Number: ACC001
Current Balance: Rs. 5000.00
==========

--- Step 2: Deposit Amount ---
Enter amount to deposit: Rs. 2000
Deposit successful! New balance: Rs. 7000.00

--- Step 3: View Account After Deposit ---

========= ACCOUNT DETAILS =========
Client Name: Khadija
CNIC: 33401-0741908-0
Account Number: ACC001
Current Balance: Rs. 7000.00
==========

--- Step 4: Attempt to Withdraw More Than Bal ance ---
Enter amount to withdraw (more than balance): Rs. 8000
Invalid Operation: Insufficient balance!
Your current balance: Rs. 7000.00

--- Step 5: Withdraw Valid Amount ---
Enter amount to withdraw (valid amount): Rs. 3000
Withdrawal successful! Amount withdrawn: Rs. 300 0.00
Remaining balance: Rs. 4000.00

--- Step 6: Final Account View ---

========= ACCOUNT DETAILS =========
Client Name: Khadija
CNIC: 33401-0741908-0
Account Number: ACC001
Current Balance: Rs. 4000.00
==========

Client 1 operations completed.